

LAPORAN TUGAS
IF3170 Inteligensi Artifisial

Implementasi Algoritma Pembelajaran Mesin



Disusun oleh :

Richard Christian	13523024
Kenneth Poenadi	13523040
Ivan Wirawan	13523046
Bob Kunanda	13523086
Muhammad Zahran Ramadhan A.	13523104

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESHA 10, BANDUNG 40132
2025

Daftar Isi

Daftar Isi.....	1
BAB I	
Logistic Regression.....	2
A. Penjelasan Singkat.....	2
B. Data Cleaning and Preprocessing.....	2
C. Penjelasan Implementasi.....	4
BAB II	
Support Vector Machine.....	11
A. Penjelasan Singkat.....	11
B. Data Cleaning and Preprocessing.....	11
C. Penjelasan Implementasi.....	12
BAB III	
Decision Tree Learning.....	20
A. Penjelasan Singkat.....	20
B. Data Cleaning and Preprocessing.....	20
C. Penjelasan Implementasi.....	22
BAB IV	
Perbandingan Algoritma.....	40
A. Perbandingan Logistic Regression.....	40
B. Perbandingan Support Vector Machine.....	42
C. Perbandingan Decision Tree Learning.....	44
BAB VII	
Penutup.....	47
A. Kesimpulan.....	47
B. Kontribusi Anggota.....	47
Lampiran.....	49

BAB I

Logistic Regression

A. Penjelasan Singkat

Mirip dengan regresi linear, konsep dari regresi logistik adalah memperkirakan nilai variabel Y berdasarkan masukan variabel X. Nilai dari variabel X sendiri merupakan variabel yang sudah diketahui, sedangkan variabel Y merupakan nilai target yang diinginkan. Dalam memperkirakan nilai variabel Y, terdapat persamaan yang didapatkan melalui berbagai data latih kumpulan nilai X dan Y. Pada dasarnya, melalui data latih, didapatkan persamaan yang paling sesuai untuk masukan X sehingga mendapatkan nilai Y. Akan tetapi, perbedaan terbesar adalah hasil dari regresi logistik. Tujuan dari regresi logistik adalah memprediksi probabilitas suatu kejadian yang bersifat biner. Jadi, hasil variabel Y akan dibandingkan dengan sebuah batas yang akan mengkategorikan nilai Y sebagai suatu kelas.

Regresi logistik memperkirakan kelas target dengan probabilitas :

$$p = P(y = 1|x, b) = \frac{1}{1 + e^{-bx}}$$

Regresi ini mengambil Fungsi Sigmoid sebagai basis perhitungannya. Dalam sebuah kurva, fungsi ini akan mengubah sebuah garis lurus menjadi berbentuk “S”. Melalui penerapan Fungsi Sigmoid ini, nilai yang dihasilkan akan selalu berada pada rentang 0 hingga 1. Nilai yang dihasilkan tersebut akan dibandingkan dengan batasan atau *threshold* yang diterapkan untuk membedakan nilai kelas target.

B. Data Cleaning and Preprocessing

- Feature Dropping

Feature dropping dilakukan untuk membuang variabel yang menambah noise atau tidak berkontribusi terhadap hasil prediksi. Untuk aplikasi Logistic Regression , dilakukan feature dropping pada variabel Marital status, Application order, International, Unemployment rate.

- One Hot Encoding

Kolom kategorikal di *encode* menggunakan OneHotEncoder melalui fungsi `one_hot_fit_transform(df)` dan `one_hot_transform(df, encoder)` yang misalnya

akan mengubah Target "Dropout" menjadi 0 atau "Graduate" menjadi 1 dan lainnya.

- Class Imbalance Handling

Berdasarkan analisis didapatkan bahwa distribusi kelas sangat tidak seimbang, untuk menangani masalah ini dilakukan oversampling menggunakan SMOTE dengan $k_neighbours = 5$. Hasilnya membuat semua kelas yang lebih kecil dari kelas terbesar menjadi sama dalam jumlah.

- Feature Engineering

Ditambahkan fitur-fitur baru yang merupakan hasil pengolahan fitur yang sudah ada untuk mendapatkan fitur yang lebih berarti terhadap hasil prediksi. Berikut adalah fitur-fitur tambahan yang digunakan untuk Logistic Regression:

- Age Grouping: membagi usia mahasiswa menjadi beberapa kelompok berdasarkan rentang usia tertentu, lalu mengonversinya menjadi tipe data numerik.

Semester 1 Metrics:

- Persentase Lulus (S1_Pass_Rate): Rasio jumlah mata kuliah yang disetujui terhadap jumlah mata kuliah yang diambil di semester 1.
- Persentase Kehadiran Evaluasi (S1_Evaluation_Attendance_Rate): Rasio jumlah evaluasi yang dihadiri terhadap jumlah mata kuliah yang diambil di semester 1.
- Persentase Menghindari Evaluasi (S1_Avoidance_Rate): 1 dikurangi persentase kehadiran evaluasi di semester 1.

Semester 2 Metrics:

- Persentase Lulus (S2_Pass_Rate): Rasio jumlah mata kuliah yang disetujui terhadap jumlah mata kuliah yang diambil di semester 2.
- Persentase Kehadiran Evaluasi (S2_Evaluation_Attendance_Rate): Rasio jumlah evaluasi yang dihadiri terhadap jumlah mata kuliah yang diambil di semester 2.
- Persentase Menghindari Evaluasi (S2_Avoidance_Rate): 1 dikurangi persentase kehadiran evaluasi di semester 2.

Progress Metrics:

- Pass Rate Progress: Perbedaan persentase lulus antara semester 2 dan semester 1.
- Grade Progress: Perbedaan nilai rata-rata antara semester 2 dan semester 1.
- Attendance Progress: Perbedaan persentase kehadiran evaluasi antara semester 2 dan semester 1.
- Avoidance Progress: Perbedaan persentase menghindari evaluasi antara semester 2 dan semester 1.

Data Cleaning:

- Mengganti nilai inf dan -inf dengan 0.
 - Mengisi nilai NaN dengan 0 untuk kolom numerik
- Normalization

Pada Logistic Regression, dilakukan normalisasi terhadap data menggunakan MinMaxScaler dan StandardScaler. Kedua teknik scaling ini dilakukan guna menyamakan skala seluruh fitur, sehingga gradient descent dapat berjalan lebih stabil, konvergen lebih cepat, serta mencegah fitur tertentu yang memiliki rentang nilai besar mendominasi proses pembelajaran model. MinMaxScaler membantu meratakan distribusi fitur ke rentang $[0,1]$, sedangkan StandardScaler membuat data memiliki mean = 0 dan standard deviation = 1, yang sangat ideal untuk model linear seperti Logistic Regression agar memperoleh decision boundary yang optimal.

C. Penjelasan Implementasi

Logistic Regression disini mencakup dua kelas:

1. LogisticRegression: Model dasar untuk klasifikasi biner, dilatih menggunakan Gradient Descent dengan dukungan untuk regularization (L1, L2, Elastic Net).
2. MulticlassLogisticRegression: Model multikelas yang menggunakan strategi One-vs-Rest dengan menginstansiasi beberapa model LogisticRegression biner.

Kelas LogisticRegression adalah inti dari model, dirancang untuk memprediksi probabilitas keluaran biner (0 atau 1).

1. Fungsi sigmoid (σ)

Fungsi ini mengubah output linier dari model menjadi probabilitas antara 0 dan 1.

$$\frac{1}{1 + e^{-b^T x}}$$

```
def sigmoid(self, z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))
```

2. Fungsi Loss (Binary Cross-Entropy / Log-Loss)

Fungsi yang dioptimalkan dalam Logistic Regression. Formula dasar untuk Binary Cross-Entropy (BCE) adalah:

$$J(\mathbf{w}, b) = -\frac{1}{N} \sum_{i=1}^N \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right] + \text{Regularization Term}$$

Dalam kode, di metode `_compute_loss`, terdapat regularization yang ditambahkan:

1. L2 Regularization (Ridge)

$$\frac{1}{2C \cdot N} \sum_{j=1}^M w_j^2$$

2. L1 Regularization (Lasso):

$$\frac{1}{2C \cdot N} \sum_{j=1}^M |w_j|$$

3. C adalah invers dari kekuatan regularisasi. C kecil berarti regularisasi kuat.

```

class LogisticRegression:
    """Binary logistic regression with gradient descent
    and sample weights"""

    def __init__(self, lr=0.01, n_iters=1000,
penalty='l2', C=1.0, random_state=42):
        self.lr = lr
        self.n_iters = n_iters
        self.penalty = penalty
        self.C = C
        self.random_state = random_state
        self.weights = None
        self.bias = None
        self.weight_history = []
        self.bias_history = []
        self.loss_history = []

```

- self.lr (*Learning Rate*): Menentukan seberapa besar langkah yang diambil pada setiap iterasi Gradient Descent saat memperbarui bobot. Nilai default adalah 0.01.
- self.n_iters (*Number of Iterations*): Jumlah perulangan pelatihan (epoch) Gradient Descent. Nilai default adalah 1000.
- self.penalty (Regularization Type): Jenis regularisasi yang digunakan. Pilihan yang didukung dalam implementasi lengkap meliputi 'l2' (default), 'l1', 'elasticnet', atau 'none'.
- self.C (*Inverse of Regularization Strength*): Parameter C dalam scikit-learn-style, yang merupakan invers dari λ (kekuatan regularisasi). Nilai C yang lebih kecil berarti regularisasi yang lebih kuat. Nilai default adalah 1.0.
- self.weights (w) dan self.bias (b): Parameter model yang akan dipelajari. Diinisialisasi sebagai None dan akan disetel ke nol di fit.
- self.loss_history, self.weight_history, self.bias_history: Digunakan untuk menyimpan nilai loss dan parameter di setiap iterasi, yang sangat berguna untuk visualisasi (seperti animasi yang Anda buat) dan *debugging*.

```

def fit(self, X, y, sample_weight=None):
    X = np.asarray(X)
    y = np.asarray(y)
    n_samples, n_features = X.shape
    self.weights = np.zeros(n_features)
    self.bias = 0

    if sample_weight is None:
        sample_weight = np.ones(n_samples)
    else:
        sample_weight = np.asarray(sample_weight)
        sample_weight = np.clip(sample_weight, 1e-8,
None)

    self.loss_history = []
    self.bias_history = []
    self.weight_history = []

    effective_lr = self.lr

    for iteration in range(self.n_iters):
        # Save current state BEFORE update

self.weight_history.append(self.weights.copy())
        self.bias_history.append(self.bias)

        # Compute loss for current parameters
        loss = self._compute_loss(X, y, self.weights,
self.bias)
        self.loss_history.append(loss)

        linear = np.dot(X, self.weights) + self.bias
        predictions = self.sigmoid(linear)

        residual = (predictions - y) * sample_weight

        dw = (1.0 / n_samples) * np.dot(X.T, residual)
        reg_term =
self._compute_regularization(self.weights, n_samples)
        dw += reg_term

        db = (1.0 / n_samples) * np.sum(residual)

        self.weights -= effective_lr * dw
        self.bias -= effective_lr * db

    # Save final state
    self.weight_history.append(self.weights.copy())
    self.bias_history.append(self.bias)
    loss = self._compute_loss(X, y, self.weights,

```



```
self.bias)
    self.loss_history.append(loss)
```

Fungsi `fit()` ini melakukan proses training Logistic Regression menggunakan algoritma gradient descent. Pada tahap awal, fungsi ini menginisialisasi bobot dan bias, kemudian menjalankan iterasi sebanyak `n_iters` untuk secara bertahap memperbaiki parameter model. Dalam setiap iterasi, model menghitung nilai loss berdasarkan bobot dan bias saat itu, menentukan gradien terhadap parameter menggunakan selisih antara prediksi dan label sebenarnya, lalu memperbarui bobot dan bias agar loss semakin menurun. Selain itu, fungsi ini juga menyimpan riwayat bobot, bias, dan loss pada setiap iterasi sehingga proses konvergensi dapat dianalisis maupun divisualisasikan. Dengan demikian, `fit()` berperan sebagai inti dari mekanisme pembelajaran Logistic Regression yang mengoptimalkan parameter secara bertahap hingga mencapai hasil terbaik.

```
class MulticlassLogisticRegression:
    """One-vs-Rest Multiclass Logistic Regression with
    support for class_weight"""

    def __init__(self, lr=0.01, n_iters=1000,
penalty='l2', C=1.0,
                    class_weight="balanced",
random_state=67):
        self.lr = lr
        self.n_iters = n_iters
        self.penalty = penalty
        self.C = C
        self.classifiers = {}
        self.classes = None
        self.class_weight = class_weight
        self.random_state = random_state
```

Kelas `MulticlassLogisticRegression` ini berfungsi untuk mengadaptasi model Regresi Logistik Biner (yang hanya bisa membedakan dua kelas) agar dapat menangani masalah klasifikasi dengan tiga kelas atau lebih (multikelas) menggunakan strategi One-vs-Rest (OvR). Kelas ini bertindak sebagai *wrapper* atau pengelola yang melatih serangkaian model LogisticRegression biner, di mana setiap model biner dilatih untuk membedakan satu kelas target melawan semua kelas lainnya.

```
def _compute_class_weights(self, y):
    labels, counts = np.unique(y, return_counts=True)
    if self.class_weight == "balanced":
        n_samples = len(y)
        n_classes = len(labels)
        weights = {lab: n_samples / (n_classes * cnt)
for lab, cnt in zip(labels, counts)}
        return weights
    elif isinstance(self.class_weight, dict):
        return self.class_weight
    else:
        return {lab: 1.0 for lab in labels}
```

Penghitungan Bobot Kelas (`_compute_class_weights`): Metode ini sangat penting untuk menangani ketidakseimbangan data (*imbalanced data*). Ketika `class_weight` disetel ke *balanced*, metode ini secara otomatis menghitung *weight* (bobot) untuk setiap kelas berdasarkan frekuensi kemunculannya. Bobot ini dirancang untuk memberikan penalti yang lebih besar pada kesalahan klasifikasi untuk kelas minoritas, memastikan model tidak mengabaikan kelas-kelas yang jarang muncul.

```
def predict(self, X):
    X_arr = X.values if isinstance(X, pd.DataFrame)
else np.asarray(X)
    probs = np.zeros((X_arr.shape[0],
len(self.classes)))

    for idx, cls in enumerate(self.classes):
        probs[:, idx] =
self.classifiers[cls].predict_proba(X_arr)

    return self.classes[np.argmax(probs, axis=1)]
```

Metode `predict(self, X)` adalah langkah pengambilan keputusan akhir dalam klasifikasi multikelas menggunakan strategi One-vs-Rest (OvR) yang diimplementasikan di kelas `MulticlassLogisticRegression`. Pertama, metode ini menyiapkan *array probs* untuk menyimpan hasil probabilitas dari semua model biner. Kemudian, ia mengiterasi melalui setiap kelas (`self.classes`), mengambil model biner yang sesuai dari `self.classifiers[cls]`, dan menggunakan metode `predict_proba()` untuk mendapatkan probabilitas bahwa sampel data `X` termasuk

dalam kelas target tersebut (ini adalah $(P \text{ Class} = \text{cls} \mid X)$). Setelah semua probabilitas dari semua model biner dikumpulkan di *array probs*, fungsi `np.argmax(probs, axis=1)` digunakan untuk menemukan indeks kelas yang memiliki nilai probabilitas tertinggi untuk setiap sampel. Indeks ini kemudian dipetakan kembali ke label kelas yang sebenarnya (`self.classes`) untuk menghasilkan prediksi akhir bagi setiap sampel.

Hierarchical Clustering

Berdasarkan analisis confusion matrix F1, didapatkan bahwa kelemahannya ada pada Enrollment yang rentan untuk diklasifikasikan sebagai Graduate dan sebaliknya. Untuk mengatasi hal ini implementasi hierarchical clustering dapat membantu untuk memfokuskan model. Hierarchical clustering dilakukan dengan prediksi siswa mana yang dropout lalu sisanya barulah diprediksi enrolled atau graduate. Implementasi ini membantu model meningkatkan F1 score.

Hyperparameter-tuning

Tuning sangatlah mempengaruhi model sehingga diperlukan tuning yang tepat. Untuk pencarian tuning yang tepat diperlakukan dua cara yaitu GridSearchCV dan RandomSearchCV dan didapatkan tuning yang tepat yaitu sebagai berikut:

lr	0.1
n_iters	13000
C	0.156
penalty	elasticnet

Tuning yang didapatkan memastikan bahwa hasil merupakan yang terbaik dan terakurat.

Stratified K-Fold

Teknik ini digunakan untuk melihat seberapa stabil model, dan memastikan bahwa pembagian setiap kelas untuk setiap fold itu seimbang.

BAB II

Support Vector Machine

A. Penjelasan Singkat

Support Vector Machine (SVM) adalah algoritma supervised learning untuk klasifikasi yang bekerja dengan mencari hyperplane pemisah terbaik antara dua kelas. SVM memodelkan keputusan sebagai fungsi linear $f(x) = w^T x + b$ dan memilih hyperplane yang memaksimalkan margin, yaitu jarak antara *hyperplane* dengan data terdekat dari masing-masing kelas. Data yang berada tepat pada batas margin ini disebut support vectors, dan hanya merekalah yang menentukan bentuk model akhir sehingga model lebih sederhana serta memiliki kemampuan generalisasi lebih baik. Untuk kasus data yang tidak dapat dipisahkan secara linear, SVM menambahkan *slack variable* agar model tetap toleran terhadap *noise* dan menggunakan kernel trick (seperti linear, polynomial, RBF, sigmoid) untuk memetakan data ke ruang berdimensi lebih tinggi sehingga menjadi linearly separable tanpa perlu menghitung transformasinya secara eksplisit. Untuk data dengan banyak kelas, SVM diperluas menggunakan strategi One-Against-All, One-Against-One, atau DAG-SVM. Secara keseluruhan, tujuan utama SVM adalah menghasilkan classifier yang stabil, robust terhadap noise, dan memiliki margin terbesar demi generalisasi optimal.

B. Data Cleaning and Preprocessing

- Feature Dropping

Feature dropping dilakukan untuk membuang variabel yang menambah noise atau tidak berkontribusi terhadap hasil prediksi. Untuk aplikasi SVM, dilakukan feature dropping pada variabel Nationality, Marital status, Application order, Gender, Scholarship holder, Unemployment rate, Admission grade, dan Father's occupation.

- Feature Encoding

Encoding dilakukan untuk mengkonversi nilai-nilai dari yang berbentuk kategori menjadi bentuk kolom-kolom biner dengan representasi numerik agar bisa

diproses oleh SVM. Dilakukan one-hot encoding untuk setiap variabel kategorikal, kecuali untuk target dimana mapping dilakukan menjadi 'Dropout': 0, 'Graduate': 1, 'Enrolled': 2.

- Class Imbalance Analysis

Dilakukan juga analisis class imbalance, tetapi pada implementasi akhir tidak dilakukan data resampling karena dataset sudah cukup balanced.

- Feature Engineering

Ditambahkan fitur-fitur baru yang merupakan hasil pengolahan fitur yang sudah ada untuk mendapatkan fitur yang lebih berarti terhadap hasil prediksi. Berikut adalah fitur-fitur tambahan yang digunakan untuk SVM:

- Total Approval, yang berisi informasi jumlah mata kuliah yang disetujui
- AvgGrade, yang berisi informasi nilai rata-rata dari 2 semester.
- ApprovalRate, yang berisi informasi persentase kelulusan mata kuliah dari matakuliah yang diambil
- AcademicPerformance, yang menggabungkan ApprovalRate dengan AvgGrade, untuk menggambarkan performa akademis mahasiswa.]

C. Penjelasan Implementasi

SVM linear dengan klasifikasi biner diimplementasikan dalam kelas LinearSVMFromScratch.

```
class LinearSVMFromScratch:
    def __init__(self, C=1.0, learning_rate=0.001,
n_iterations=1000, random_state=None):
        self.C = C
        self.learning_rate = learning_rate
        self.n_iterations = n_iterations
        self.random_state = random_state
        self.w = None # weights
        self.b = None # bias
        self.losses = []
```

Inisialisasi parameter utama dari kelas.

C adalah parameter yang digunakan untuk menyesuaikan seberapa sensitif model terhadap kesalahan prediksi

Learning_rate menentukan seberapa besar langkah atau perubahan pada parameter model untuk setiap iterasi pelatihan.

N_iterations menentukan berapa banyak iterasi yang dilakukan dalam pelatihan model.

w dan b adalah parameter yang akan diisi oleh model untuk melakukan pembelajaran.

Losses menyimpan perubahan nilai loss untuk melakukan analisis konvergensi model dari setiap iterasi.

```
def _hinge_loss(self, X, y):
    n_samples = X.shape[0]
    reg_loss = 0.5 * np.dot(self.w, self.w)

    margins = y * (np.dot(X, self.w) + self.b)
    hinge_loss = self.C * np.sum(np.maximum(0, 1 -
margins))

    total_loss = reg_loss + hinge_loss
    return total_loss / n_samples
```

Hinge loss digunakan untuk mengukur loss keseluruhan dari model, dengan menggabungkan regularization loss dengan margins.

Regularization loss digunakan untuk memastikan bobot model tidak terlalu besar, sehingga menjaga hasil model untuk tetap bersifat general dan mencegah overfitting.

Margin digunakan untuk mengukur seberapa jauh titik dari hyperplane, bila margin bernilai > 1 , maka titik sudah cukup jauh, sedangkan semakin kecil margin yang dihasilkan maka semakin buruk hasil prediksi.

Kedua perhitungan ini memastikan agar hasil model akhir tidak terlalu overfit, sekaligus memaksimalkan hasil prediksi.

```
def fit(self, X, y, verbose=False):
    if self.random_state is not None:
```

```

        np.random.seed(self.random_state)

        n_samples, n_features = X.shape

        self.w = np.zeros(n_features)
        self.b = 0

        for iteration in range(self.n_iterations):
            margins = y * (np.dot(X, self.w) + self.b)

            misclassified_mask = margins < 1

            dw = self.w - self.C * np.dot(X.T,
misclassified_mask * y)
            db = -self.C * np.sum(misclassified_mask * y)

            self.w -= self.learning_rate * dw
            self.b -= self.learning_rate * db

            loss = self._hinge_loss(X, y)
            self.losses.append(loss)

            if verbose and (iteration + 1) % 100 == 0:
                print(f"  Iteration {iteration +
1}/{self.n_iterations}, Loss: {loss:.6f}")

        return self

```

Fit adalah fungsi utama yang dilakukan untuk melakukan training pada model svm. W disini menyimpan arah dari pemisah, sedangkan b menyimpan posisinya. Model diinisialisasi dengan nilai 0 sebelum dilakukan pelatihan.

Model lalu melakukan pelatihan dalam loop for iteration sebanyak iterasi yang sudah

ditentukan. Setiap iterasi akan melakukan update pada w dan b model sebesar learning rate yang sudah ditentukan.

Untuk mengetahui seberapa besar untuk menyesuaikan w dan b , model akan mengukur seberapa jauh margin untuk dataset terlebih dahulu. Klasifikasi yang bermasalah dengan margin < 1 kemudian akan digunakan untuk menghitung gradien baru.

Perhitungan bobot w dan b yang baru kemudian dilakukan secara matematis dengan memperhitungkan C secara gradient descent. Dalam setiap iterasi, svm akan sedikit demi sedikit menggeser w dan b untuk meminimalisir loss yang didapatkan sekaligus menjaga model tetap sederhana.

```
def predict(self, X):
    decision = np.dot(X, self.w) + self.b
    return np.sign(decision)
```

Prediksi lalu dilakukan dengan mengecek lokasi titik terhadap hyperplane.

Implementasi SVM lalu dilanjutkan dalam kelas `MultiClassSVMFromScratch`, yang menambahkan fitur untuk melakukan multi-class classification. Pada kelas ini, diimplementasikan dua strategi untuk klasifikasi multi-class yaitu One-vs-One (OvO), dan One-vs-Rest (OvR).

```
class MultiClassSVMFromScratch:
    def __init__(self, strategy='ovo', C=1.0,
learning_rate=0.001, n_iterations=1000, random_state=None):
        self.strategy = strategy
        self.C = C
        self.learning_rate = learning_rate
        self.n_iterations = n_iterations
        self.random_state = random_state
        self.classifiers = []
        self.classes_ = None
```



```
self.class_pairs = []
```

Inisialisasi utama dari kelas multiclass. Inisialisasi serupa dengan LinearSVM, dengan tambahan strategi, yang merupakan pilihan antara ovo dan ovr, classifiers yang menyimpan daftar model yang dilatih, classes_ yang menyimpan label kelas unik, dan class_pairs yang digunakan oleh ovo saja untuk menyimpan pasangan antar kelas yang dilatih.

```
def fit(self, X, y, verbose=False):
    self.classes_ = np.unique(y)
    n_classes = len(self.classes_)

    if self.strategy == 'ovo':
        for i in range(n_classes):
            for j in range(i + 1, n_classes):
                class_i, class_j = self.classes_[i],
self.classes_[j]

                mask = (y == class_i) | (y == class_j)
                X_pair = X[mask]
                y_pair = y[mask]

                y_binary = np.where(y_pair == class_i, -1,
1)

                clf = LinearSVMFromScratch(
                    C=self.C,
                    learning_rate=self.learning_rate,
                    n_iterations=self.n_iterations,
                    random_state=self.random_state
                )
                clf.fit(X_pair, y_binary, verbose=verbose)

                self.classifiers.append(clf)
```

```
self.class_pairs.append((class_i,
class_j))
```

Fit disini membagi proses pelatihan sesuai dengan strategi yang dipilih (ovo atau ovr). Pada bagian ini, diimplementasikan strategi ovo dimana program akan mulai dengan mengambil daftar kelas unik, kemudian melakukan looping antara setiap pasangan kelas. Bagian mask lalu memastikan bahwa dataset yang diberikan pada LinearSVM hanya berisi 2 kelas yang sudah terpilih, dan label juga diubah menjadi -1 dan 1 agar kompatibel dengan LinearSVM. Proses pelatihan lalu dilakukan oleh LinearSVM, dimana hasil model biner lalu disimpan dalam classifiers dan pasangan yang digunakan dalam pelatihan model disimpan dalam class_pairs.

```
elif self.strategy == 'ovr':
    for class_label in self.classes_:
        y_binary = np.where(y == class_label, 1, -1)

        clf = LinearSVMFromScratch(
            C=self.C,
            learning_rate=self.learning_rate,
            n_iterations=self.n_iterations,
            random_state=self.random_state
        )
        clf.fit(X, y_binary, verbose=verbose)

        self.classifiers.append(clf)

    return self
```

Implementasi ovr sedikit berbeda dengan implementasi ovo, dimana program hanya perlu membuat 1 model untuk masing-masing kelas. Pelabelan data diubah juga menjadi -1 dan 1, tetapi -1 berisi seluruh kelas selain kelas target yang sedang dilatih.

```
def predict(self, X):
    n_samples = X.shape[0]
```

```

        if self.strategy == 'ovo':
            votes = np.zeros((n_samples, len(self.classes_)))

            for clf, (class_i, class_j) in
zip(self.classifiers, self.class_pairs):
                predictions = clf.predict(X)

                class_i_idx = np.where(self.classes_ ==
class_i)[0][0]
                class_j_idx = np.where(self.classes_ ==
class_j)[0][0]

                votes[predictions == -1, class_i_idx] += 1
                votes[predictions == 1, class_j_idx] += 1

            predicted_indices = np.argmax(votes, axis=1)
            return self.classes_[predicted_indices]

        elif self.strategy == 'ovr':
            decision_values = np.zeros((n_samples,
len(self.classes_)))

            for i, clf in enumerate(self.classifiers):
                decision_values[:, i] =
clf.decision_function(X)

            predicted_indices = np.argmax(decision_values,
axis=1)

            return self.classes_[predicted_indices]

```

Prediksi juga menyesuaikan untuk masing-masing ovo dan ovr.

Pada implementasi ovo, kelas prediksi ditentukan dengan menggunakan sistem suara

atau voting. Program memulai dengan membuat tabel suara yang awalnya masih kosong. Prediksi lalu dilakukan dengan looping setiap pasangan kelas dan mengisi tabel voting. Bila nilai prediksi negatif, maka class i mendapatkan 1 suara, sedangkan bila bernilai positif maka class j mendapatkan satu suara. Argmax lalu memilih kolom dengan suara terbesar yang diambil sebagai pemenang dari proses voting.

Pada implementasi ovr, classifier memilih dengan membandingkan decision value dari setiap model. Semakin besar decision value, maka classifier tersebut semakin yakin terhadap hasilnya. Proses serupa dengan ovo, tetapi tabel hanya menyimpan decision value dari setiap classifier dan mengambil kelas dengan nilai tertinggi.

BAB III

Decision Tree Learning

A. Penjelasan Singkat

Decision Tree Learning (DTL) adalah salah satu metode *machine learning* yang mengakpromasi hasil nilai target *value* yang bernilai diskrit. Makna dari nilai diskrit adalah variabel yang memiliki kemungkinan beberapa nilai terbatas. Secara singkat, *decision tree learning* akan merepresentasikan proses pemilihan melalui berbagi aturan *if-else* dalam bentuk *decision tree*.

Setiap *node* pada *decision tree* merepresentasikan sebuah tes untuk atribut. Kemudian, setiap *node* akan memiliki beberapa akar yang menunjukkan kemungkinan nilai dari atribut tersebut. Setiap nilai akan tersambung lagi kepada *node* lain untuk atribut yang berbeda. *Node* daun atau *node* terakhir merepresentasikan hasil dari klasifikasi.

Terdapat beberapa jenis *decision tree learning* seperti *Iterative Dichotomiser 3* (ID3), C4.5, dan *Classification and Regression Tree* (CART). Algoritma ID3 bersifat *top-down* dengan *greedy search*. Pada setiap *node*, algoritma akan memilih atribut terbaik yang mampu memberikan pembagian paling jelas. Maksudnya adalah perbedaan atribut ini akan menyebabkan perbedaan kelas dengan jumlah yang cukup signifikan. Sedangkan algoritma C4.5 dibuat untuk mendukung adanya nilai atribut yang tunggal seperti tanggal. Algoritma ini akan menghitung nilai *gain ratio* sebagai kriteria dalam melakukan *split*. Algoritma CART menggunakan *gini impurity* yang menunjukkan seberapa tidak murni sebuah distribusi kelas pada *node*.

B. Data Cleaning and Preprocessing

- Feature Dropping

Dalam algoritma *Decision Tree Learning* juga dilakukan penghapusan beberapa atribut yang dirasa tidak relevan dalam prediksi. Tujuannya adalah mengurangi dimensi data sekaligus mencegah adanya *overfitting*. Dalam kasus ini, beberapa fitur yang dihapus adalah *marital status*, *international*, *nationality*, *mother's qualification*, *father's qualification*, *gender* dan *unemployment rate*. Dengan penghapusan beberapa fitur ini, *noise* pada data akan berkurang sehingga pembelajaran menjadi lebih baik.

- Feature Encoding

Selain penghapusan beberapa fitur, kolom kategorikal akan di-*encode* menggunakan *One Hot Encoding* yang akan mengubah datanya menjadi representasi numerik. Alasan penggunaan *One Hot Encoding* daripada *Label Encoding* karena tidak ada deskripsi hirarki yang jelas antara nilai dari atribut.

- Imbalance Class Analysis

Dalam menangani adanya imbalansi antar nilai kelas, maka kelas dengan data yang lebih sedikit (sampel lebih sedikit) akan diberikan *weight* lebih besar. Maknanya adalah model akan lebih mementingkan kelas tersebut yang merupakan minoritas. Dengan ini, bisa ke kelas mayoritas akan bisa dikurangi dan memberikan performa lebih baik pada seluruh kelas.

- Feature Engineering

Untuk membantu mesin menangani pembelajaran lebih baik, maka terdapat beberapa fitur baru yang dibuat yang merupakan gabungan dari fitur-fitur awal. Dengan adanya fitur ini, diharapkan mesin mendapatkan atribut yang lebih baik untuk membagi data ke dalam kelas-kelasnya. Salah satu contohnya adalah fitur `TotalApproval` yang merupakan `Curricular_units_1st_sem_approved + Curricular_units_2nd_sem_approved`. Contoh lain adalah `ApprovalRate` yang merupakan `TotalApproval/TotalEnrolled`. Beberapa fitur baru lagi adalah `TotalEnrolled` (total beban akademik yang diambil), `TotalEvaluation` (total evaluasi yang diikuti), `ApprovalDifference` (total kelas yang diambil dan di terima), `Academic_load_risk` (mengukur unit yang gagal), `AvgGrade` (rata-rata nilai), `Grade_Consistency` (kekonsistenan nilai), `AcademicPerformance` (kombinasi perkalian kelulusan dan nilai), `AcademicPerformancePlus` (mirip dengan `AcademicPerformance` namun menggunakan penjumlahan), `AcademicPerformaceTimesEvaluation` (mahasiswa yang rajin evaluasi dan memiliki nilai bagus), `Financial_Risk_Score` (interpretasi kondisi finansial), `Grade_Improvement` (kondisi dinamika nilai).

C. Penjelasan Implementasi

Algoritma *Decision Tree Learning* (DTL) yang diimplementasikan adalah C4.5.

Seluruh algoritma ini berada di bawah kelas `DecisionTreeC45`. Adapun beberapa fungsi di dalamnya adalah :

```
class Node:
    def __init__(self, feature=None, threshold=None,
left=None, right=None, value=None, gain=None,
split_type='binary', children=None, split_values=None)

def is_leaf(self):
    return self.value is not None
```

Kelas `Node` ini merupakan pembentuk *Decision Tree*. Setiap `Node` memiliki atribut `feature` (fitur yang digunakan untuk *split*), `threshold` (nilai batas untuk *binary split*), `left` (anak kiri), `right` (anak kanan), `value` (label kelas untuk node daun), `gain` (gain ratio), `split_type` (*binary* atau *multiway*), `children` (untuk *multiway*) dan `split_values` (list nilai untuk *multiway*). Fungsi `is_leaf()` akan mengembalikan node dengan `value` bukan `None`.

```
Class DecisionTreeC45 :
    def __init__(self, max_depth=None,
min_samples_split=2, min_samples_leaf=1, class_weight=None,
enable_multiway_splits=True, max_bins=6)
```

Untuk kelas `DecisionTree45` dibentuk dari beberapa atribut yaitu `max_depth` (kedalaman maksimum tree), `min_samples_split` (minimum jumlah sampel untuk *split*), `min_samples_leaf` (minimum sampel di node daun), `class_weight` (*dictionary weight* untuk *imbalance class*), `enable_multiway_splits` (pengaktifan *multiway splits*), `max_bins` (jumlah *bins* untuk diskretisasi maksimum)

```
def entropy(self, y):
    classes, counts = np.unique(y, return_counts=True)

    if self.class_weight is not None:
        weighted_counts = [counts[i] *
self.class_weight.get(classes[i], 1.0)
for i in range(len(classes))]
        probabilities = weighted_counts /
weighted_counts.sum()
    else:
        probabilities = counts / counts.sum()

    return -np.sum([p * np.log2(p) for p in probabilities
if p > 0])
```

Fungsi ini akan menghitung nilai entropi dari kelas target. Di dalam fungsi ini, ditambahkan juga pembobotan kelas untuk jumlah sampel yang lebih kecil. Setelah penyesuaian dengan bobot, maka entropi dari probabilitas akan dihitung.

```
def information_gain(self, y, y_left, y_right):
    parent_entropy = self.entropy(y)
    n = len(y)
    n_left, n_right = len(y_left), len(y_right)

    child_entropy = (n_left / n) * self.entropy(y_left) +
                    (n_right / n) * self.entropy(y_right)

    return parent_entropy - child_entropy
```

Fungsi ini akan menghitung nilai *information gain* dari atribut. Hal ini digunakan untuk menunjukkan seberapa baik split pada fitur ini.

```
def gain_ratio(self, y, y_left, y_right):
    ig = self.information_gain(y, y_left, y_right)
    n = len(y)
    n_left, n_right = len(y_left), len(y_right)

    # Split Info
    si = -(n_left/n) * np.log2(n_left/n) - (n_right/n) *
np.log2(n_right/n)

    return ig / si if si > 0 else 0
```

Fungsi ini digunakan untuk fitur dengan banyak *unique value* agar mengurangi *overfitting*. Fungsi ini akan menormalisasi *information gain* dengan nilai *split info*.

```
def split_info(self, splits, y):
    n = len(y)
    split_info_value = 0

    for split_data in splits.values():
        if len(split_data) == 0:
            continue
        p = len(split_data) / n
        if p > 0:
            split_info_value -= p * np.log2(p)

    return split_info_value
```

Fungsi ini menghitung nilai persebaran data dalam atribut yang memiliki banyak *unique value*. Algoritmanya bekerja dengan menghitung proporsi data yang masuk ke

setiap cabang lalu entropi akan dihitung berdasarkan proporsi tersebut.
<pre>def information_gain_multiway(self, y, splits): parent_entropy = self.entropy(y) n = len(y) weighted_entropy = 0 for split_data in splits.values(): if len(split_data) == 0: continue weight = len(split_data) / n weighted_entropy += weight * self.entropy(split_data) return parent_entropy - weighted_entropy</pre>
Fungsi ini menghitung <i>information gain</i> untuk <i>multiway split</i> .
<pre>def gain_ratio_multiway(self, y, splits): ig = self.information_gain_multiway(y, splits) si = self.split_info(splits, y) return ig / si if si > 0 else 0</pre>
Fungsi ini akan menghitung nilai <i>gain ratio</i> untuk <i>multiway split</i> . Rumus yang digunakan masih sama yaitu <i>information gain</i> dibagi dengan <i>split info</i> . Namun, fungsi ini dikhususkan untuk pencegahan bias terhadap <i>split</i> yang memiliki banyak cabang
<pre>def discretize_feature(self, X_column, y): result = self.find_best_split_points(X_column, y, max_splits=self.max_bins - 1) if result is None: return None bins, bin_edges, gain = result if len(bins) <= 2: return None return bins, bin_edges</pre>
Fungsi ini akan melakukan diskritisasi untuk fitur yang <i>continuous</i> . Dengan ini, nilai <i>continuous</i> akan dikelompokkan menjadi beberapa <i>bins</i> .
<pre>def find_best_split_points(self, X_column, y,</pre>

```

max_splits=4):
    # cari titik split terbaik dengan entropy
    unique_values = np.unique(X_column)

    if len(unique_values) <= 2:
        return None

    sorted_indices = np.argsort(X_column)
    X_sorted = X_column[sorted_indices]
    y_sorted = y[sorted_indices]

    candidate_points = []
    for i in range(len(unique_values) - 1):
        split_point = (unique_values[i] +
unique_values[i + 1]) / 2
        candidate_points.append(split_point)

    if len(candidate_points) == 0:
        return None

        # Try different numbers of splits (2 to
max_splits)
        best_splits = None
        best_splits = None
        best_gain = -1

        for n_splits in range(2, min(max_splits + 1,
len(candidate_points) + 1)):
            split_points =
self._find_n_best_splits(X_sorted, y_sorted, n_splits)

            if split_points is None or len(split_points)
== 0:
                continue

            regions =
self._create_regions_from_splits(X_column, y,
split_points)
            gain = self.information_gain_multiway(y,
regions)

            if gain > best_gain:
                best_gain = gain
                best_splits = split_points

        if best_splits is None or len(best_splits) == 0:

```

```

        return None

        # Create final bins with labels
        bins, bin_edges =
self._create_bins_from_splits(X_column, y, best_splits)
        return bins, bin_edges, best_gain

```

Fungsi ini akan mencari titik potong terbaik untuk fitur yang kontinu dengan menggunakan entropi. Algoritma ini akan menggenerasi beberapa kandidat, lalu mencoba beberapa konfigurasi. Hasilnya adalah konfigurasi dengan information gain terbanyak.

```

def _find_n_best_splits(self, X_sorted, y_sorted,
n_splits):
    # cari n titik split terbaik dengan reduksi
entropy
    if n_splits <= 0 or len(X_sorted) < n_splits + 1:
        return None

    split_points = []
    regions = [(0, len(X_sorted))]

    for _ in range(n_splits):
        best_split_pos = None
        best_split_value = None
        best_entropy_reduction = -1
        best_region_idx = None

        for region_idx, (start, end) in
enumerate(regions):
            if end - start < 2 *
self.min_samples_leaf:
                continue

            region_y = y_sorted[start:end]
            region_X = X_sorted[start:end]

            for i in range(start +
self.min_samples_leaf, end - self.min_samples_leaf):
                if X_sorted[i] == X_sorted[i - 1]:
                    continue

                left_y = y_sorted[start:i]
                right_y = y_sorted[i:end]

                parent_entropy =
self.entropy(region_y)

```

```

        n = len(region_y)
        weighted_entropy = (len(left_y) / n) *
self.entropy(left_y) + \
                                (len(right_y) / n) *
self.entropy(right_y)
        entropy_reduction = parent_entropy -
weighted_entropy

        if entropy_reduction >
best_entropy_reduction:
            best_entropy_reduction =
entropy_reduction
            best_split_pos = i
            best_split_value = (X_sorted[i -
1] + X_sorted[i]) / 2
            best_region_idx = region_idx

        if best_split_pos is None:
            break

        split_points.append(best_split_value)

        old_start, old_end = regions[best_region_idx]
        regions[best_region_idx] = (old_start,
best_split_pos)
        regions.insert(best_region_idx + 1,
(best_split_pos, old_end))

    return sorted(split_points) if len(split_points) >
0 else None

```

Fungsi ini akan mencari beberapa titik split terbaik dengan *greedy* untuk fitur yang bersifat kontinu. Algoritma ini akan melakukan pencarian secara iteratif yang mencari split untuk memaksimalkan reduksi entropi. Kemudian, akan dilakukan split dengan reduksi entropi yang paling besar hingga mendapatkan sejumlah n.

```

def _create_regions_from_splits(self, X_column, y,
split_points):
    # buat regions dari split points
    regions = {}
    split_points_sorted = sorted(split_points)

    mask = X_column <= split_points_sorted[0]
    regions[f"<={split_points_sorted[0]:.2f}"] =
y[mask]

    for i in range(len(split_points_sorted) - 1):

```

```

        mask = (X_column > split_points_sorted[i]) &
        (X_column <= split_points_sorted[i + 1])

regions[f"({split_points_sorted[i]:.2f},{split_points_sorted[i+1]:.2f})"] = y[mask]

        mask = X_column > split_points_sorted[-1]
        regions[f">{split_points_sorted[-1]:.2f}"] = y[mask]

    return regions

```

Fungsi ini akan membuat *dictionary regions* dari split points. Algoritma ini akan membagi menjadi 3 region yaitu yang pertama (kurang dari nilai split[0]), tengah (diantara split[i] hingga split[i+1]) dan terakhir (lebih besar dari split[n-1])

```

def _create_bins_from_splits(self, X_column, y,
split_points):
    # buat bins dan edges dari split points
    split_points_sorted = sorted(split_points)
    bins = self._create_regions_from_splits(X_column,
y, split_points_sorted)
    bin_edges = [X_column.min()] + split_points_sorted
+ [X_column.max()]
    return bins, np.array(bin_edges)

```

Fungsi ini akan membentuk *bins* dan *bin edges* dari *split points*. Membuat beberapa region dan menjadikannya *bin edges*.

```

def discretize_feature(self, X_column, y):
    # diskretisasi fitur continuous dengan multi-way
    splitting
    result = self.find_best_split_points(X_column, y,
max_splits=self.max_bins - 1)

    if result is None:
        return None

    bins, bin_edges, gain = result

    if len(bins) <= 2:
        return None

    return bins, bin_edges

```

Fungsi ini akan melakukan diskretisasi fitur kontinu untuk menjadi kategori diskrit. Algoritma ini akan memanggil fungsi `find_best_split_points()` dan mengembalikan bins

jika jumlahnya lebih dari 2.

```
def best_split(self, X, y):
    best_gain = -1
    best_feature = None
    best_threshold = None
    best_splits = None
    best_bin_edges = None
    best_split_type = 'binary'

    n_features = X.shape[1]

    if self.enable_multiway_splits:
        multiway_feature, multiway_splits,
        multiway_edges, multiway_gain, is_categorical = \
            self.best_multiway_split(X, y)

        if multiway_splits is not None and
        len(multiway_splits) >= 3:
            best_gain = multiway_gain
            best_feature = multiway_feature
            best_splits = multiway_splits
            best_bin_edges = multiway_edges
            best_split_type = 'multiway'

    for feature_idx in range(n_features):
        X_column = X[:, feature_idx]
        thresholds = np.unique(X_column)

        for threshold in thresholds:
            left_mask = X_column <= threshold
            right_mask = ~left_mask

            if np.sum(left_mask) <
            self.min_samples_leaf or np.sum(right_mask) <
            self.min_samples_leaf:
                continue

            y_left = y[left_mask]
            y_right = y[right_mask]
            gain = self.gain_ratio(y, y_left, y_right)

            if gain > best_gain:
                best_gain = gain
                best_feature = feature_idx
                best_threshold = threshold
```

```

        best_splits = None
        best_bin_edges = None
        best_split_type = 'binary'

    return best_feature, best_threshold, best_splits,
    best_bin_edges, best_gain, best_split_type

```

Fungsi ini mencari titik split terbaik untuk semua fitur baik itu *binary* dan *multi-way*. Algoritmanya akan mencoba melakukan binary splits untuk semua fitur dan batas, lalu gain ratio setiap kandidat akan dihitung. Setelah itu, fitur dan split terbaik akan dikembalikan sebagai hasil.

```

def best_multiway_split(self, X, y):
    best_gain = -1
    best_feature = None
    best_splits = None
    best_bin_edges = None
    best_is_categorical = False

    n_features = X.shape[1]

    for feature_idx in range(n_features):
        X_column = X[:, feature_idx]
        n_unique = len(np.unique(X_column))

        is_categorical = n_unique <= 20
        if is_categorical and
self.enable_multiway_splits:
            unique_values = np.unique(X_column)

            if len(unique_values) <= 1:
                continue

            splits = {}
            for val in unique_values:
                mask = X_column == val
                splits[f"={val:.0f}"] = y[mask]
                if any(len(split_y) <
self.min_samples_leaf for split_y in splits.values()):
                    continue
                    continue
            gain = self.gain_ratio_multiway(y, splits)

            if gain > best_gain:
                best_gain = gain
                best_feature = feature_idx

```

```

        best_splits = splits
        best_bin_edges = unique_values
        best_is_categorical = True

        if not is_categorical and
self.enable_multiway_splits:
            discretize_result =
self.discretize_feature(X_column, y)

            if discretize_result is not None:
                bins, bin_edges = discretize_result

                if any(len(bin_y) <
self.min_samples_leaf for bin_y in bins.values()):
                    continue
                    continue
                    gain = self.gain_ratio_multiway(y,
bins)

                if gain > best_gain:
                    best_gain = gain
                    best_feature = feature_idx
                    best_splits = bins
                    best_bin_edges = bin_edges
                    best_is_categorical = False

            return best_feature, best_splits, best_bin_edges,
best_gain, best_is_categorical

```

Fungsi ini akan mencari *multi-way split* terbaik dari semua fitur. Untuk fitur kategorikal, algoritma akan memecah pada setiap nilai unik. Sedangkan untuk fitur kontinu akan di diskretisasi terlebih dahulu. Terakhir, split dengan gain ratio tertinggi akan dipilih.

```

def build_tree(self, X, y, depth=0):
    n_samples, n_features = X.shape
    n_classes = len(np.unique(y))

    if depth >= self.max_depth or n_classes == 1 or
n_samples < self.min_samples_split:
        leaf_value = self.most_common_label(y)
        return Node(value=leaf_value)

    best_feature, best_threshold, best_splits,
best_bin_edges, best_gain, split_type = \

```



```

        self.best_split(X, y)

        if best_feature is None:
            leaf_value = self.most_common_label(y)
            return Node(value=leaf_value)

        if split_type == 'binary':
            # Binary split
            left_mask = X[:, best_feature] <=
best_threshold
            right_mask = ~left_mask

            left_child = self.build_tree(X[left_mask],
y[left_mask], depth + 1)
            right_child = self.build_tree(X[right_mask],
y[right_mask], depth + 1)

            return Node(feature=best_feature,
threshold=best_threshold,
                        left=left_child, right=right_child,
gain=best_gain, split_type='binary')

        else:
            # Multi-way split
            children = {}
            X_column = X[:, best_feature]

            for split_label in best_splits.keys():
                mask = None

                if '=' in split_label:
                    # Categorical split: =value
                    val = float(split_label.split('=')[1])
                    mask = X_column == val

                elif split_label.startswith('<='):
                    # First region: <=x
                    val = float(split_label[1:])
                    mask = X_column <= val

                elif split_label.startswith('>'):
                    # Last region: >x
                    val = float(split_label[1:])
                    mask = X_column > val

                elif '(' in split_label and ')' in

```

```

split_label:
    # Middle region: (x,y]
    range_str =
split_label.strip('()[]').split(',')
    lower = float(range_str[0])
    upper = float(range_str[1])
    mask = (X_column > lower) & (X_column
<= upper)

    elif '[' in split_label and ')' in
split_label:
    # Old format support: [x,y)
    range_str =
split_label.strip('[]()').split(',')
    lower = float(range_str[0])
    upper = float(range_str[1])

    # Find index of this bin
    bin_idx =
list(best_splits.keys()).index(split_label)

    if bin_idx == len(best_splits) - 1:
        mask = (X_column >= lower) &
(X_column <= upper)
    else:
        mask = (X_column >= lower) &
(X_column < upper)

    if mask is not None and np.sum(mask) > 0:
        child = self.build_tree(X[mask],
y[mask], depth + 1)
        children[split_label] = child

    return Node(feature=best_feature,
gain=best_gain, split_type='multiway',
children=children,
split_values=list(best_splits.keys()))

```

Fungsi ini akan membangun *decision tree* secara rekursif. Algoritma akan terus membentuk pohon hingga *max depth* atau semua kelas sama atau sampel terlalu dikit. Split akan dilakukan dengan fungsi `best_split`. Kemudian, jika split binary, maka akan dibentuk 2 anakan. Sedangkan untuk multiway, akan dibentuk sebanyak *n* anakan.

```

def most_common_label(self, y):
    if self.class_weight is not None:
        classes, counts = np.unique(y,
return_counts=True)

```

```

        weighted_counts = np.array([counts[i] *
self.class_weight.get(classes[i], 1.0)
                                for i in
range(len(classes))])
        return classes[np.argmax(weighted_counts)]
    else:
        return np.bincount(y).argmax()

```

Fungsi ini akan menentukan label mayoritas untuk node daun. Jika ada `class_weight`, maka kelas yang bobotnya tertinggi akan dipilih. Jika tidak maka akan dipilih kelas dengan count tertinggi.

```

def fit(self, X, y):
    if isinstance(X, pd.DataFrame):
        self.feature_names = X.columns.tolist()
        X = X.values
    else:
        self.feature_names = [f"feature_{i}" for i in
range(X.shape[1])]

    self.n_features = X.shape[1]
    self.root = self.build_tree(X, y)
    return self

```

Fungsi untuk melakukan *training* kepada model.

```

def predict_sample(self, x, node):
    if node.is_leaf():
        return node.value

    if node.split_type == 'binary':
        # Binary split
        if x[node.feature] <= node.threshold:
            return self.predict_sample(x, node.left)
        else:
            return self.predict_sample(x, node.right)

    else:
        # Multi-way split
        X_val = x[node.feature]

        # Find matching child
        for split_label, child in
node.children.items():
            if '=' in split_label:
                # Categorical split
                val = float(split_label.split('=')[1])

```

```

        if X_val == val:
            return self.predict_sample(x,
child)

        elif split_label.startswith('<='):
            # First region: ≤x
            val = float(split_label[1:])
            if X_val <= val:
                return self.predict_sample(x,
child)

        elif split_label.startswith('>'):
            # Last region: >x
            val = float(split_label[1:])
            if X_val > val:
                return self.predict_sample(x,
child)

        elif '(' in split_label and ']' in
split_label:
            # Middle region: (x,y]
            range_str =
split_label.strip('() []').split(',')
            lower = float(range_str[0])
            upper = float(range_str[1])
            if lower < X_val <= upper:
                return self.predict_sample(x,
child)

        elif '[' in split_label:
            # Old format support: [x,y)
            range_str =
split_label.strip('[ ] ()').split(',')
            lower = float(range_str[0])
            upper = float(range_str[1])

            # Check if last bin (includes upper
bound)
            bin_idx =
list(node.children.keys()).index(split_label)

            if bin_idx == len(node.children) - 1:
                if lower <= X_val <= upper:
                    return self.predict_sample(x,
child)

            else:

```

```

        if lower <= X_val < upper:
            return self.predict_sample(x,
child)

        # Default: return most common label if no
match found
        return node.value if hasattr(node, 'value')
and node.value is not None else 0

```

Fungsi ini adalah fungsi tes yang akan memprediksi satu sampel dengan penelusuran *decision tree*. Masukan akan ditelusuri melalui percabangan pohon hingga mendapatkan leaf yang berisikan nilai prediksi.

```

def predict(self, X):
    if isinstance(X, pd.DataFrame):
        X = X.values
    return np.array([self.predict_sample(x, self.root)
for x in X])

```

Fungsi ini juga merupakan fungsi tes yang memprediksi banyak sampel sekaligus. Hal ini akan memprediksi setiap sampel dengan fungsi `predict_sample()`

```

def print_tree(self, node=None, depth=0, prefix="",
is_last=True, parent_prefix=""):
    if node is None:
        node = self.root
        print("\n" + "="*80)
        print("DECISION TREE STRUCTURE")
        print("="*80)

    # Create tree branch visualization
    if depth == 0:
        branch = "Root"
    else:
        branch = parent_prefix + ("└─ " if is_last
else "├─ ")

    if node.is_leaf():
        print(f"{branch} Leaf: Class={node.value}")
    elif node.split_type == 'binary':
        feature_name =
self.feature_names[node.feature] if self.feature_names
else f"X[{node.feature}]"
        print(f"{branch} {feature_name} <=
{node.threshold:.4f} (Gain: {node.gain:.4f})")

```

```

        # Prepare prefix for children
        child_prefix = parent_prefix + ("    " if
is_last else "|    ") if depth > 0 else ""

        if node.left and node.right:
            self.print_tree(node.left, depth + 1, "<
True ", False, child_prefix)
            self.print_tree(node.right, depth + 1, "<
False ", True, child_prefix)
        elif node.left:
            self.print_tree(node.left, depth + 1, "<
True ", True, child_prefix)
        elif node.right:
            self.print_tree(node.right, depth + 1, "<
False ", True, child_prefix)
        else:
            # Multi-way split
            feature_name =
self.feature_names[node.feature] if self.feature_names
else f"X[{node.feature}]"
            print(f"{branch} {feature_name} (Multi-way,
{len(node.children)} branches, Gain: {node.gain:.4f})")

            child_prefix = parent_prefix + ("    " if
is_last else "|    ") if depth > 0 else ""

            children_list = list(node.children.items())
            for i, (split_label, child) in
enumerate(children_list):
                is_last_child = (i == len(children_list) -
1)
                self.print_tree(child, depth + 1,
f"{split_label} ", is_last_child, child_prefix)

            if depth == 0:
                print("="*80)

```

Fungsi pencetak visualisasi struktur pohon secara rekursif menggunakan format ASCII. Selain itu, ditambahkan beberapa informasi tambahan seperti fitur dan gain.

```

def get_depth(self, node=None):
    if node is None:
        node = self.root

    if node.is_leaf():
        return 0

```

```

        if node.split_type == 'binary':
            left_depth = self.get_depth(node.left) if
node.left else 0
            right_depth = self.get_depth(node.right) if
node.right else 0
            return 1 + max(left_depth, right_depth)
        else:
            # Multi-way split
            max_child_depth = 0
            for child in node.children.values():
                child_depth = self.get_depth(child)
                max_child_depth = max(max_child_depth,
child_depth)
            return 1 + max_child_depth

```

Fungsi penghitung kedalaman maksimum pohon dengan melakukan rekursi. Basisnya adalah leaf node dengan kedalaman 0

```

def count_nodes(self, node=None):
    if node is None:
        node = self.root

    if node.is_leaf():
        return 1

    if node.split_type == 'binary':
        left_count = self.count_nodes(node.left) if
node.left else 0
        right_count = self.count_nodes(node.right) if
node.right else 0
        return 1 + left_count + right_count
    else:
        # Multi-way split
        total_count = 1
        for child in node.children.values():
            total_count += self.count_nodes(child)
        return total_count

```

Fungsi penghitung jumlah *node* pada pohon secara rekursif juga dengan basis node leaf yang berjumlah 1.

```

def count_leaves(self, node=None):
    if node is None:
        node = self.root

    if node.is_leaf():
        return 1

```

```
        if node.split_type == 'binary':
            left_leaves = self.count_leaves(node.left) if
node.left else 0
            right_leaves = self.count_leaves(node.right)
if node.right else 0
            return left_leaves + right_leaves
        else:
            # Multi-way split
            total_leaves = 0
            for child in node.children.values():
                total_leaves += self.count_leaves(child)
            return total_leaves
```

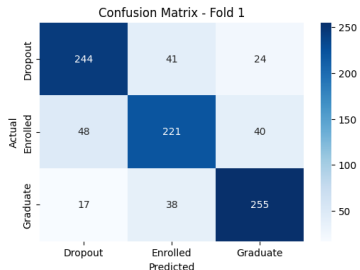
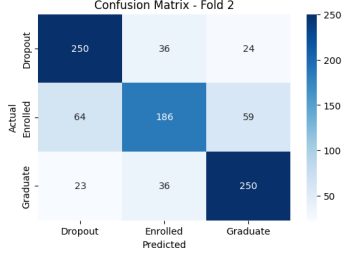
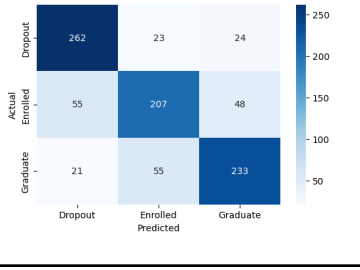
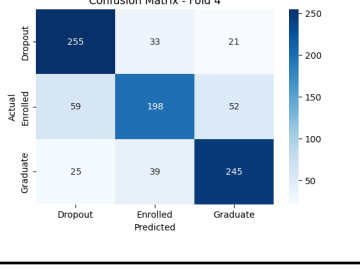
Fungsi penghitung jumlah daun pada pohon dengan rekursi penambahan dengan basis nilai daun 1.

BAB IV

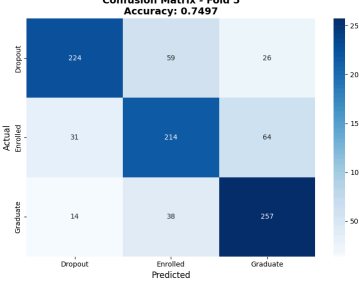
Perbandingan Algoritma

A. Perbandingan Logistic Regression

Perbandingan logistic regression dilakukan menggunakan SklearnLogisticRegression. Semua preprocessing, cleaning, proses, dan tuning disamakan dengan yang digunakan oleh model buatan. Didapatkan hasil sebagai berikut:

Model	Fold	Confusion Matrix	Nilai Akurasi (Lokal)	Nilai F1 Macro (Lokal)																
Model yang kami buat	1	Confusion Matrix - Fold 1 <table><thead><tr><th></th><th>Dropout</th><th>Enrolled</th><th>Graduate</th></tr></thead><tbody><tr><th>Dropout</th><td>244</td><td>41</td><td>24</td></tr><tr><th>Enrolled</th><td>48</td><td>221</td><td>40</td></tr><tr><th>Graduate</th><td>17</td><td>38</td><td>255</td></tr></tbody></table>		Dropout	Enrolled	Graduate	Dropout	244	41	24	Enrolled	48	221	40	Graduate	17	38	255	0.7759	0.7754
		Dropout	Enrolled	Graduate																
	Dropout	244	41	24																
	Enrolled	48	221	40																
Graduate	17	38	255																	
2	Confusion Matrix - Fold 2 <table><thead><tr><th></th><th>Dropout</th><th>Enrolled</th><th>Graduate</th></tr></thead><tbody><tr><th>Dropout</th><td>250</td><td>36</td><td>24</td></tr><tr><th>Enrolled</th><td>64</td><td>186</td><td>59</td></tr><tr><th>Graduate</th><td>23</td><td>36</td><td>250</td></tr></tbody></table>		Dropout	Enrolled	Graduate	Dropout	250	36	24	Enrolled	64	186	59	Graduate	23	36	250	0.7392	0.7359	
	Dropout	Enrolled	Graduate																	
Dropout	250	36	24																	
Enrolled	64	186	59																	
Graduate	23	36	250																	
3	Confusion Matrix - Fold 3 <table><thead><tr><th></th><th>Dropout</th><th>Enrolled</th><th>Graduate</th></tr></thead><tbody><tr><th>Dropout</th><td>262</td><td>23</td><td>24</td></tr><tr><th>Enrolled</th><td>55</td><td>207</td><td>48</td></tr><tr><th>Graduate</th><td>21</td><td>55</td><td>233</td></tr></tbody></table>		Dropout	Enrolled	Graduate	Dropout	262	23	24	Enrolled	55	207	48	Graduate	21	55	233	0.7565	0.7549	
	Dropout	Enrolled	Graduate																	
Dropout	262	23	24																	
Enrolled	55	207	48																	
Graduate	21	55	233																	
4	Confusion Matrix - Fold 4 <table><thead><tr><th></th><th>Dropout</th><th>Enrolled</th><th>Graduate</th></tr></thead><tbody><tr><th>Dropout</th><td>255</td><td>33</td><td>21</td></tr><tr><th>Enrolled</th><td>59</td><td>198</td><td>52</td></tr><tr><th>Graduate</th><td>25</td><td>39</td><td>245</td></tr></tbody></table>		Dropout	Enrolled	Graduate	Dropout	255	33	21	Enrolled	59	198	52	Graduate	25	39	245	0.7530	0.7508	
	Dropout	Enrolled	Graduate																	
Dropout	255	33	21																	
Enrolled	59	198	52																	
Graduate	25	39	245																	

	5	Confusion Matrix - Fold 5 <table><tr><th>Actual \ Predicted</th><th>Dropout</th><th>Enrolled</th><th>Graduate</th></tr><tr><th>Dropout</th><td>247</td><td>42</td><td>20</td></tr><tr><th>Enrolled</th><td>45</td><td>202</td><td>62</td></tr><tr><th>Graduate</th><td>18</td><td>40</td><td>251</td></tr></table>	Actual \ Predicted	Dropout	Enrolled	Graduate	Dropout	247	42	20	Enrolled	45	202	62	Graduate	18	40	251	0.7551	0.7538
	Actual \ Predicted	Dropout	Enrolled	Graduate																
Dropout	247	42	20																	
Enrolled	45	202	62																	
Graduate	18	40	251																	
	average		0.7542	0.7542																
SklearnLogisticRegression	1	Confusion Matrix - Fold 1 Accuracy: 0.7769 <table><tr><th>Actual \ Predicted</th><th>Dropout</th><th>Enrolled</th><th>Graduate</th></tr><tr><th>Dropout</th><td>235</td><td>48</td><td>26</td></tr><tr><th>Enrolled</th><td>24</td><td>229</td><td>56</td></tr><tr><th>Graduate</th><td>12</td><td>41</td><td>257</td></tr></table>	Actual \ Predicted	Dropout	Enrolled	Graduate	Dropout	235	48	26	Enrolled	24	229	56	Graduate	12	41	257	0.7769	0.7776
	Actual \ Predicted	Dropout	Enrolled	Graduate																
	Dropout	235	48	26																
	Enrolled	24	229	56																
Graduate	12	41	257																	
2	Confusion Matrix - Fold 2 Accuracy: 0.7457 <table><tr><th>Actual \ Predicted</th><th>Dropout</th><th>Enrolled</th><th>Graduate</th></tr><tr><th>Dropout</th><td>226</td><td>53</td><td>31</td></tr><tr><th>Enrolled</th><td>33</td><td>207</td><td>69</td></tr><tr><th>Graduate</th><td>15</td><td>35</td><td>259</td></tr></table>	Actual \ Predicted	Dropout	Enrolled	Graduate	Dropout	226	53	31	Enrolled	33	207	69	Graduate	15	35	259	0.7457	0.7450	
Actual \ Predicted	Dropout	Enrolled	Graduate																	
Dropout	226	53	31																	
Enrolled	33	207	69																	
Graduate	15	35	259																	
3	Confusion Matrix - Fold 3 Accuracy: 0.7381 <table><tr><th>Actual \ Predicted</th><th>Dropout</th><th>Enrolled</th><th>Graduate</th></tr><tr><th>Dropout</th><td>239</td><td>43</td><td>27</td></tr><tr><th>Enrolled</th><td>40</td><td>207</td><td>63</td></tr><tr><th>Graduate</th><td>17</td><td>53</td><td>239</td></tr></table>	Actual \ Predicted	Dropout	Enrolled	Graduate	Dropout	239	43	27	Enrolled	40	207	63	Graduate	17	53	239	0.7381	0.7382	
Actual \ Predicted	Dropout	Enrolled	Graduate																	
Dropout	239	43	27																	
Enrolled	40	207	63																	
Graduate	17	53	239																	
4	Confusion Matrix - Fold 4 Accuracy: 0.7368 <table><tr><th>Actual \ Predicted</th><th>Dropout</th><th>Enrolled</th><th>Graduate</th></tr><tr><th>Dropout</th><td>239</td><td>44</td><td>26</td></tr><tr><th>Enrolled</th><td>48</td><td>194</td><td>67</td></tr><tr><th>Graduate</th><td>16</td><td>43</td><td>250</td></tr></table>	Actual \ Predicted	Dropout	Enrolled	Graduate	Dropout	239	44	26	Enrolled	48	194	67	Graduate	16	43	250	0.7368	0.7352	
Actual \ Predicted	Dropout	Enrolled	Graduate																	
Dropout	239	44	26																	
Enrolled	48	194	67																	
Graduate	16	43	250																	

	5		0.7497	0.7496
	average		0.7495	0.7491

Dapat dilihat bahwa model yang kami buat sedikit lebih baik dibandingkan model SklearnLogisticRegression namun hal ini dapat disebabkan oleh karena hyperparameter tuning dilakukan pada model yang kami buat bukan model SklearnLogisticRegression sehingga mungkin tuning SklearnLogisticRegression belum optimal.

B. Perbandingan Support Vector Machine

Perbandingan Support Vector Machine dilakukan menggunakan SklearnSVC. Semua preprocessing, cleaning, proses, dan tuning disamakan dengan yang digunakan oleh model buatan. Didapatkan hasil sebagai berikut:

Implementation	Mean Accuracy (Val)	Mean F1-Score (Val)	Mean F1-Weighted (Val)
From Scratch - One-vs-One (OvO)	0.755495	0.692482	0.751107
Sklearn - One-vs-One (OvO)	0.701872	0.652690	0.709767
From Scratch - One-vs-Rest (OvR)	0.738374	0.660558	0.727314
Sklearn - One-vs-Rest (OvR)	0.668596	0.587540	0.662468

Dapat dilihat bahwa model kami menghasilkan hasil yang lebih bagus dibandingkan hasil yang dihasilkan oleh SKLearnSVC yaitu dengan perbandingan sebagai berikut:

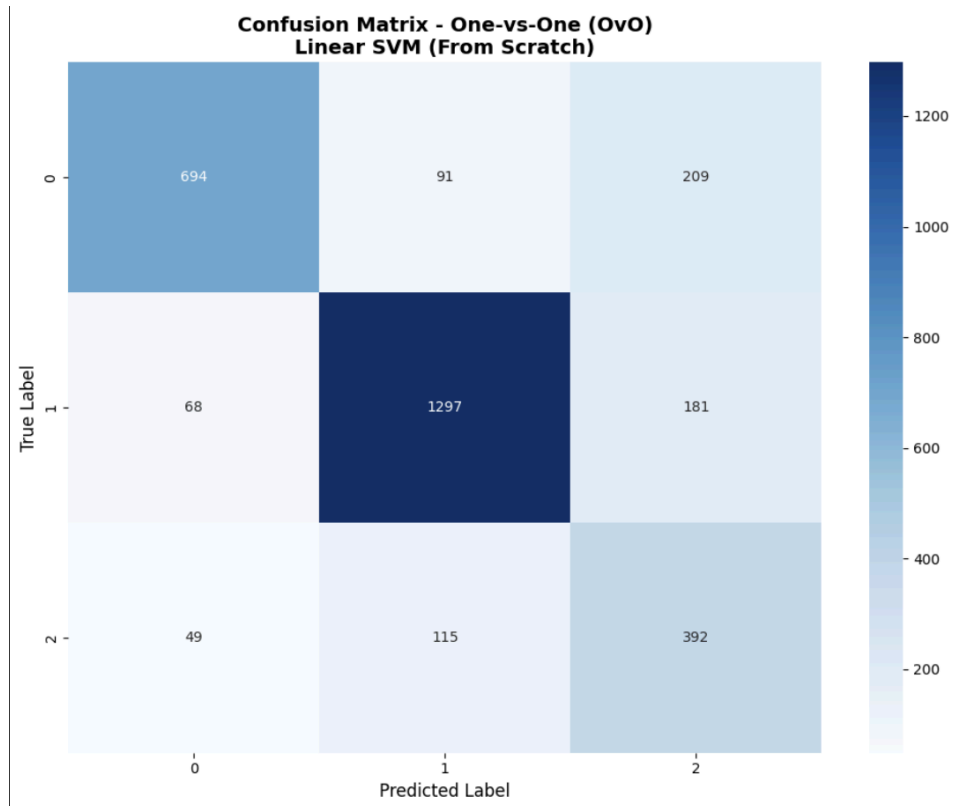
One-vs-One (OvO):

- Accuracy Difference: +5.36% (Scratch vs Sklearn)
- F1-Macro Difference: +3.98%

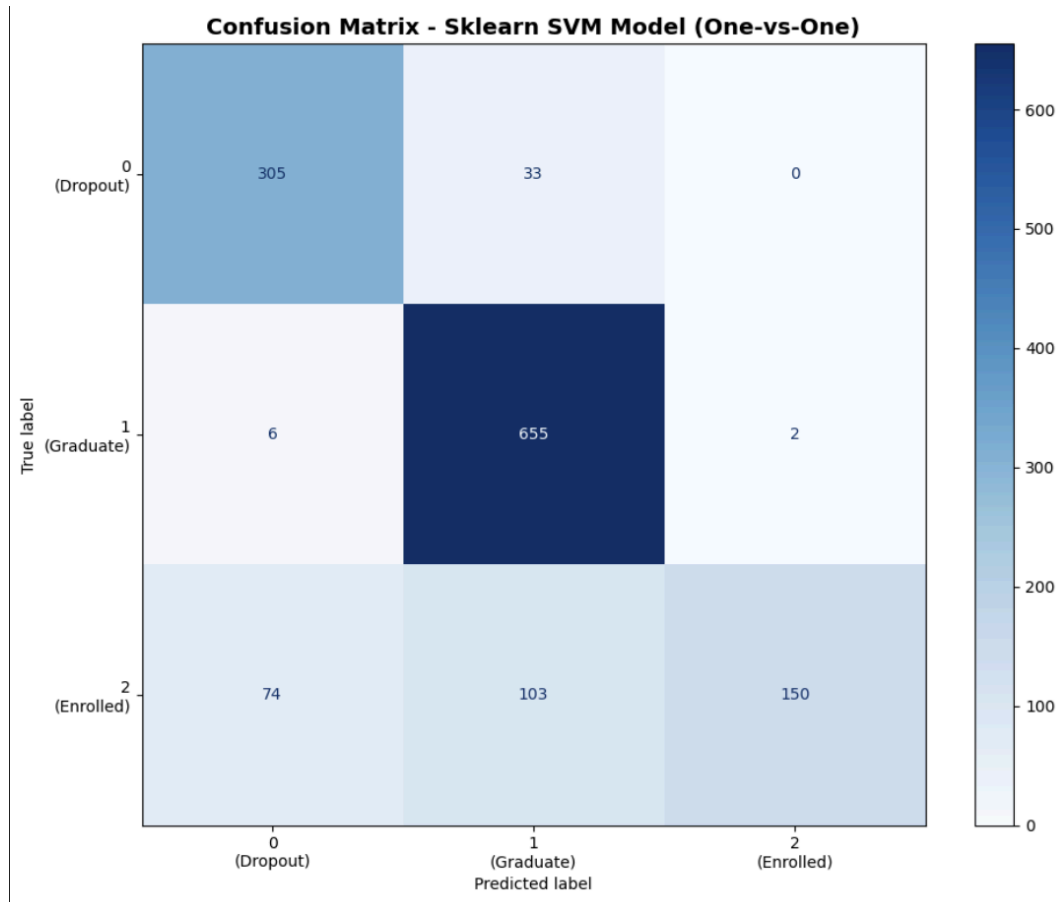
- F1-Weighted Difference: +4.13%

One-vs-Rest (OvR):

- Accuracy Difference: +6.98% (Scratch vs Sklearn)
- F1-Macro Difference: +7.30%
- F1-Weighted Difference: +6.48%



Gambar 4.1 Confusion Matriks SVM From Scratch



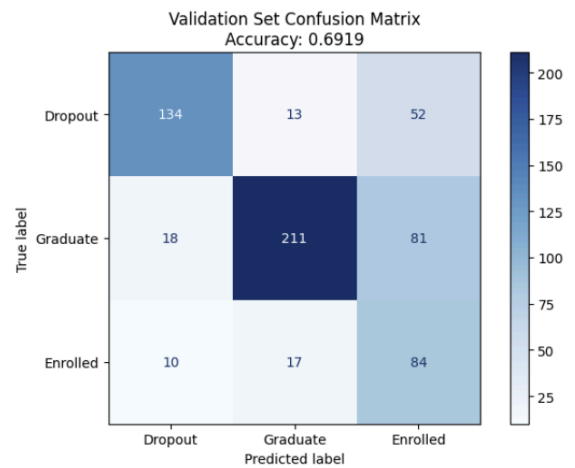
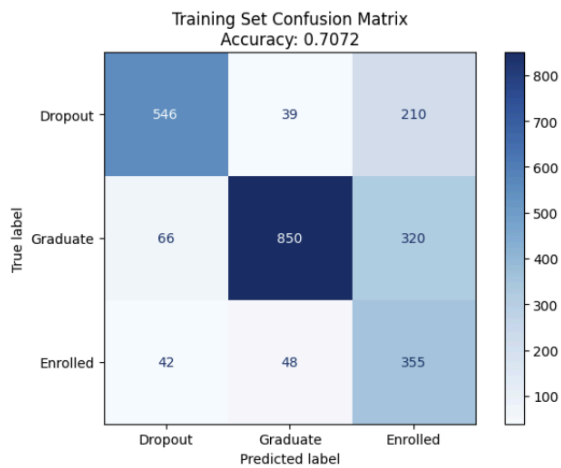
Gambar 4.2 Confusion Matriks SVM From SkicitLearn

C. Perbandingan Decision Tree Learning

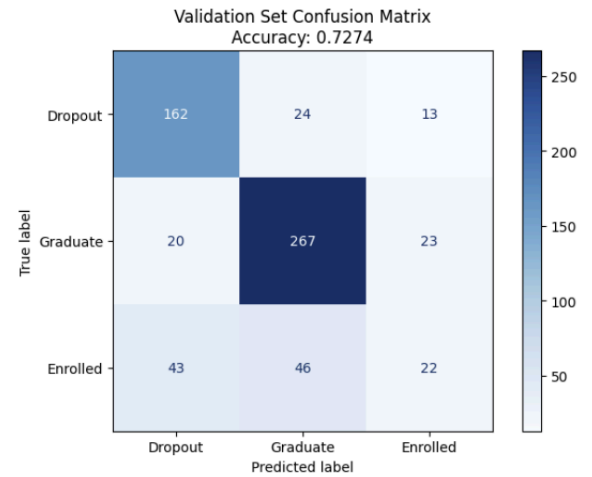
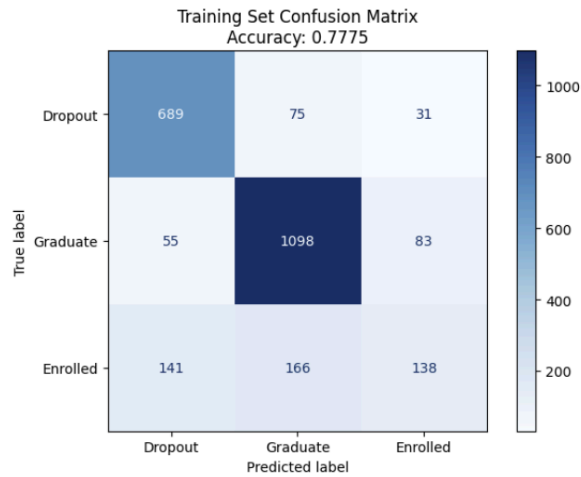
Evaluasi kinerja algoritma dilakukan dengan membandingkan model Decision Tree *from scratch* dengan implementasi standar pustaka **sklearn.tree.DecisionTreeClassifier**. Pada implementasi pustaka, parameter *class_weight='balanced'* diterapkan sebagai *baseline* untuk menangani ketidakseimbangan kelas.

Berbeda dengan pendekatan standar, model *from scratch* dirancang menggunakan strategi Hierarchical Decision Tree Learning (DTL). Strategi ini memprioritaskan pemisahan kelas target 'Dropout' dari kelas lainnya terlebih dahulu, sebelum membedakan 'Enrolled' dan 'Graduate' yang memiliki kemiripan fitur. Proses pelatihan model ini juga melibatkan *fine-tuning* ketat pada validation set dengan ruang pencarian parameter *max_depth* [4, 5], *min_samples_split* [3, 6], dan *min_samples_leaf* [2, 4]. Rentang kedalaman 6 dan 7 sengaja dieksklusi dari eksperimen untuk menghindari kecenderungan algoritma memilih model yang terlalu kompleks (overfitting)."

Confusion Matriks Sklearn Decision Tree



Confusion Matriks Decisiton Tree From Scratch



Berdasarkan perbandingan Confusion Matrix di atas, terlihat bahwa model Hierarchical DTL (from scratch) mampu menghasilkan akurasi validasi yang lebih tinggi (72.74%) dibandingkan model Sklearn (69.19%). *Insight* yang didapat dari perbandingan tersebut adalah:

1. Efektivitas Struktur Hirarkis: Strategi memisahkan 'Dropout' di awal terbukti efektif pada model scratch. Hal ini terlihat dari tingginya jumlah prediksi benar pada kelas 'Dropout' dan 'Graduate' di Validation Set. Model *scratch* sangat superior dalam mengidentifikasi kelas mayoritas 'Graduate' (267 prediksi benar) dibandingkan Sklearn (211 prediksi benar).

2. Trade-off Kelas 'Enrolled': Meskipun akurasi global model scratch lebih tinggi, model Sklearn dengan *class_weight='balanced'* terlihat lebih agresif dalam menebak kelas minoritas 'Enrolled' (84 benar vs 22 benar pada scratch), namun hal ini menyebabkan banyaknya false positive (kesalahan prediksi) dimana data 'Graduate' sering dianggap 'Enrolled' oleh Sklearn. Model scratch lebih konservatif dan stabil, meminimalkan kebingungan antar kelas dominan.
3. Kontrol Overfitting: Pembatasan *max_depth* pada rentang 4-5 sukses menjaga kemampuan generalisasi model scratch. Meskipun akurasi training mencapai 77.75%, penurunan ke akurasi validasi (72.74%) masih dalam batas wajar dan tetap lebih unggul daripada baseline pustaka. Ini menunjukkan bahwa model berhasil menangkap pola utama data tanpa terjebak menghafal noise yang mungkin terjadi jika kedalaman pohon dibiarkan mencapai level 6 atau 7."

BAB VII

Penutup

A. Kesimpulan

Melalui hasil eksperimen dengan data latih dan data uji, didapatkan kesimpulan bahwa algoritma pembelajaran mesin yang paling efektif untuk dataset tingkat kesuksesan akademik adalah *Support Vector Machine* (SVM) dengan F1 score pada public sebesar 0.73149.

Logistic regression sangat baik untuk data yang berisi mayoritas tipe data numerikal dan bukan model yang optimal untuk data seperti yang diberikan. Untuk menghadapi tipe data kategorikal diperlukan encoding dan scaling agar seluruh data berada pada range yang sama dan bertipe numerical. Logistic regression bergantung sekali pada tuning sehingga hyperparameter tuning sangat dibutuhkan agar hasil optimal.

Sementara itu, performa algoritma Decision Tree Learning (DTL) menghasilkan F1 score yang tidak setinggi model SVM ataupun Logistic Regression. Kendala utama model ini terletak pada klasifikasi kelas 'Enrolled', di mana DTL masih kesulitan untuk menarik batas pemisah (decision boundary) yang tegas antara kelas tersebut dengan kelas lainnya. Selain itu, stabilitas DTL sangat bergantung pada eksplorasi kedalaman pohon (depth exploration); pemilihan kedalaman yang kurang presisi berisiko tinggi menyebabkan underfitting atau overfitting. Oleh karena itu, model ini membutuhkan tuning yang cermat, khususnya dalam menentukan jumlah minimum sampel pada leaf sebagai mekanisme pembatasan (pruning) agar model dapat menggeneralisasi data dengan baik.

B. Kontribusi Anggota

Nama	NIM	Kontribusi
Richard Christian	13523024	Support Vector Machine (SVM) implementation, laporan
Kenneth Poenadi	13523040	Support Vector Machine (SVM) implementation, laporan

Ivan Wirawan	13523046	Logistic Regression training, laporan
Bob Kunanda	13523086	Logistic Regression training, laporan
M. Zahran Ramadhan A.	13523104	Decision Tree Learning (DTL) implementation, laporan

Lampiran

Referensi

- [1] GeeksforGeeks, "Hierarchical Clustering in Data Mining," *GeeksforGeeks*, 2021. [Online]. Available: <https://www.geeksforgeeks.org/data-science/hierarchical-clustering-in-data-mining/>
- [2] Medium, "Logistic Regression using Gradient Descent," *Medium*, 2019. [Online]. Available: <https://medium.com/intro-to-artificial-intelligence/logistic-regression-using-gradient-descent-bf8cbe749ceb>
- [3] scikit-learn Developers, "ElasticNet," *scikit-learn Documentation*, 2023. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.ElasticNet.html
- [4] GeeksforGeeks, "Support Vector Machine (SVM) Algorithm," *GeeksforGeeks*, 2020. [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/support-vector-machine-algorithm/>
- [5] scikit-learn Developers, "Support Vector Machines," *scikit-learn Documentation*, 2023. [Online]. Available: <https://scikit-learn.org/stable/modules/svm.html>
- [6] Medium, "Support Vector Machine (SVM)," *Medium*, 2019. [Online]. Available: <https://medium.com/sysinfo/support-vector-machine-svm-5d95a7d7a547>
- [7] GeeksforGeeks, "How to tune a Decision Tree in Hyperparameter tuning," *GeeksforGeeks*, Jul. 23, 2025. [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/how-to-tune-a-decision-tree-in-hyperparameter-tuning/>
- [8] Meegle, "Fine-Tuning For Decision Trees," *Meegle*, Oct. 25, 2025. [Online]. Available: https://www.meegle.com/en_us/topics/fine-tuning/fine-tuning-for-decision-trees

Tautan Repository :

Github: <https://github.com/KennethhPoenadi/AI-Shiteru/>